# Handling Data Skew in Spark

## Production-Grade Deep Dive (with real-world fixes)

Prepared By : Kaviraj T U

---

## 1️⃣ Why Data Skew Is Dangerous

In production, skew causes the classic “everything finishes... except one task” problem:

- ✅ Some tasks finish in seconds
- 🐢 One task runs for **40+ minutes**
- 💥 Executors crash
- 🧨 Jobs fail with **OOM** or excessive spilling

---

## 2️⃣ What Is Data Skew?

**Data skew = uneven data distribution across partitions.**

Spark processes:

- **1 partition → 1 task**

So if one partition becomes huge:

- That task becomes a **bottleneck**
- The entire stage waits (even if 199 tasks finished)

This is the **straggler task** problem.

---

## 3️⃣ How Skew Happens (Most Common Causes)

### 🔹 1) Skewed Join Keys (Most painful in prod)

Example distribution:

| customer_id | count |
|---|---|
| 101 | 10 rows |
| 102 | 8 rows |
| 999 | 50,000,000 rows |

If you join on `customer_id`, **all rows for** `999` **land in one partition**.

**Result**: one executor gets overloaded → **slow stage + memory spike**.

---

### 🔹 2) Poor Partition Strategy

Example:

```
df.repartition(200, "country")
```

If:

- `country = "US"` holds **90%** of data

Then most data still ends up in **a few partitions**, and tasks become uneven.

---

### 🔹 3) Aggregations on Skewed Columns

Example:

```
df.groupBy("status").count()
```

If:

- `SUCCESS` = 95% rows
- `FAILED` = 5% rows

Then aggregation becomes skewed.

---

## 4️⃣ How to Detect Data Skew (Fast & Reliable)

### 🔎 In Spark UI (best signal)

Look for:

- One task running **much longer**
- Huge difference in task duration
- One task has massive **Shuffle Read / Spill**

### 🔎 Programmatically (confirm the culprit keys)

```
(df.groupBy("customer_id").count()
 .orderBy("count", ascending=False)
 .show())
```

If **one key dominates** → skew is confirmed.

---

## 5️⃣ Real Production Scenario (What it looks like)

You join:

- `orders` = **1B rows**
- `customers` = **50M rows**
- One customer appears **200M times**

What happens:

- One partition handles ~200M rows
- Executor memory spikes
- Shuffle spills to disk
- The whole job slows down drastically

---

# 6️⃣ Production-Grade Solutions (What actually works)

## ✅ 1) Salting Technique (Best for heavy skew)

Goal: split one hot key into multiple keys so it spreads across partitions.

### Step 1: Add salt to the big table

```
from pyspark.sql.functions import rand, floor

orders_salted = orders.withColumn(
    "salt",
    floor(rand() * 10)
)
```

**Step 2: Expand the small table**

```
salt_df = spark.range(10).withColumnRenamed("id", "salt")

customers_expanded = customers.crossJoin(salt_df)
```

**Step 3: Join using salted key**

```
result = orders_salted.join(
    customers_expanded,
    ["customer_id", "salt"]
)
```

✅ Now the skewed key is distributed across **10 partitions** instead of 1.

---

## ✅ 2) Broadcast Join (When one side is small)

If `customers` is small enough:

```
from pyspark.sql.functions import broadcast

orders.join(broadcast(customers), "customer_id")
```

✅ No shuffle → skew impact becomes minimal.

---

## ✅ 3) Increase Shuffle Partitions (Quick win for moderate skew)

Default:

```
spark.sql.shuffle.partitions = 200
```

Increase:

```
spark.conf.set("spark.sql.shuffle.partitions", 500)
```

✅ More partitions → smaller data per partition

⚠️ Too high can increase overhead, so tune wisely.

---

## ✅ 4) Enable AQE (Must-have in Spark 3+)

```
spark.conf.set("spark.sql.adaptive.enabled", "true")
```

AQE can:

- Detect skewed partitions
- Split large partitions dynamically
- Improve join strategies automatically

---

# 7️⃣ Strategy Comparison (Quick Cheat Sheet)

| Method | Best for | Complexity |
|---|---|---|
| Broadcast Join | Small lookup table | Easy |
| Increase partitions | Moderate skew | Easy |
| Salting | Heavy skew / hot keys | Medium |
| AQE | Spark 3+ production | Easy |

---

## 8️⃣ Interview Questions (Quick Answers)

**Q1: What is data skew in Spark?**

Uneven distribution across partitions causing slow tasks and bottlenecks.

**Q2: How do you detect skew?**

Spark UI task time imbalance + large shuffle read/spill, then confirm with key distribution checks.

**Q3: How do you fix skewed joins?**

Salting, broadcast join, increasing shuffle partitions, and enabling AQE.

---

## 9️⃣ Real-World Insight (What many teams miss)

- A large percentage of Spark performance issues are skew-related
- Most people scale the cluster first
- Very few fix **partition imbalance**

---

## 🔟 Key Takeaways (Save this)

- ✔️ Skew = slow stages, OOM risk, spills, stragglers
- ✔️ Find it in Spark UI (task duration + shuffle size)
- ✔️ Salting is powerful for hot keys
- ✔️ Broadcast avoids shuffle when possible
- ✔️ AQE is a production must-have (Spark 3+)

- ✔️ Always analyze key distribution before joining

---

*Prepared by: Kaviraj T U*

## Thank You.!

## Happy Learning ...